Fachhochschule Dortmund

University of Applied Sciences and Arts

Projektarbeit 1

Einarbeitung in das hls4ml Framework zur FPGA-basierten Machine-Learning-Inferenz

Vorgelegt von:

Herrn Simo Gueno Yvan Valder

Betreuer: Prof. Dr. Michael Karagounis

Datum: April 2026

Kurzzusammenfassung

Die vorliegende Projektarbeit beschäftigt sich mit der Einarbeitung in das Open-Source-Framework hls4ml (High Level Synthesis for Machine Learning), das die automatisierte Übersetzung von trainierten Maschine-Learning-Modellen in synthetisierbaren HLS-Code für FPGAs ermöglicht. Im Mittelpunkt stehen die theoretischen Grundlagen von FPGAs, die Kernkonzepte des Frameworks (Optimierungsstrategien, Parallelismus, Pipelining, Reuse Factor, Precision, Profiling und Pruning) sowie die praktische Durchführung des offiziellen hls4ml-Tutorials (Part 1 bis Part 8) auf dem PYNQ-Z1-Board. Abschließend werden Ressourcenkennzahlen verglichen, aufgetretene Hürden, ein Fazit und ein Ausblick auf weiterführende Arbeiten präsentiert.

Abstract

This project focuses on familiarizing oneself with the open-source framework hls4ml (High Level Synthesis for Machine Learning), which enables the automated translation of trained machine learning models into synthesizable HLS code for FPGAs. The focus is on the theoretical fundamentals of FPGAs, the core concepts of the framework (optimization strategies, parallelism, pipelining, reuse factor, precision, profiling and pruning), as well as the practical implementation of the official hls4ml tutorial (Parts 1 to 8) on the PYNQ-Z1 board. Finally, resource metrics are compared, obstacles encountered are discussed, and a conclusion and outlook on further work are presented.

Inhaltsverzeichnis

Kurzzusammenfassung ... 2
Inhaltsverzeichnis ... 3
1 Einleitung ... 5
2 Theorie ... 6
2.1 FPGA – Grundlagen und Bezug zu hls4ml ... 6
2.1.1 Evolution des FPGA-Designs: HLS und hls4ml ... 6
2.1.2 PYNQ-Z1 – Zielplattform ... 6
2.2 HLS4ML-Frameworks ... 7
2.2.1 Parallelismus und Pipeline ... 7
2.2.2 Reuse Factor ... 7
2.2.3 Precision – Festkommadarstellung ... 8
2.2.4 Profiling und Trace ... 8
2.2.5 Pruning und Sparsity ... 8
2.2.6 Latency und Initiation Interval (II) ... 8
2.2.7 Granularität der Quantisierung ... 8
2.3 Umgebung – Vorbereitung und Kompatibilität ... 9
2.3.1 Software-Stack und Conda-Umgebung ... 9
2.3.2 Auswahl und Begründung der Linux-Entwicklungsumgebung? ... 9
3 Praktische Durchführung des HLS4ML-Tutorials ... 9
3.1 Part 1_Getting_started (Grundlegende hls4ml-Übersetzung) ... 9
3.1.1 Analyse der Modellspezifikation und Präzision ... 10
3.1.2 Analyse des Latenz-Reports (Tabelle 3) ... 10
3.1.3 Analyse der Ressourcenauslastung (Tabelle 4) ... 10
3.2 Part 2_Advanced_Configuration (Profiling, Präzisionsoptimierung, Reuse Factor und Ressourcenoptimierung) ... 11
3.2.1 Analyse der Modellspezifikation und Präzision ... 11
3.2.2 Analyse des Latenz-Reports (Tabelle 5) ... 12
3.2.3 Analyse der Ressourcenauslastung (Tabelle 6) ... 12
3.3 Part 3_Compression (Pruning und Sparsity) ... 13
3.3.1 Analyse der Modellspezifikation und Präzision ... 13
3.3.2 Analyse des Latenz-Reports (Tabelle 7) ... 13
3.3.3 Analyse der Ressourcenauslastung (Tabelle 8) ... 13
3.4 Part 4_Quantization (Quantisierung mit QKeras) ... 14

3.4.1 Analyse der Modellspezifikation und Präzision 14
3.4.2 Analyse des Latenz-Reports (Tabelle 9) 14
3.4.3 Analyse der Ressourcenauslastung (Tabelle 10) 15
3.5 Part 4.1_HG_Quantization () 15
3.5.1 Analyse der Modellspezifikation und Präzision 15
3.5.2 Analyse des Latenz-Reports (Tabelle 11) 16
3.5.3 Analyse der Ressourcenauslastung (Tabelle 12) 16
3.6 Part 5_BDT (Conifer: Entscheidungsbäume auf FPGAs) 17
3.6.1 Analyse der Modellspezifikation und Präzision 17
3.6.2 Inferenz auf dem Pynq-Z1 (Abbildung 1) 17
3.7 Part 6_CNNs (Convolution auf FPGAs) 18
3.7.1 Analyse der Modellspezifikation und Präzision 18
3.7.2 Analyse des Latenz-Reports 18
3.7.3 Analyse der Ressourcenauslastung 19
3.8 Part 7 – PYNQ-bitstream generierung, Deployment und validation 19
7a - Part7a_Bitstream 19
7b - Part7b_Deployment 21
7c - Part7c_Validation 21
3.9 Part 8_Symbolic_Regression 22
3.9.1 Analyse der Modellspezifikation und Präzision 22
3.9.2 Vergleich der Inferenz-Ausgaben 22
3.9.3 Fazit zur symbolischen Regression 22
3.10 Zusammenfassung der Ergebnisse 23
4 Hürden bei der Umsetzung 24
4.1 Systemkompatibilität 24
4.2 Vitis-Backend-Problem: Versionskonflikt 24
4.3 PYNQ-Z1 vs. PYNQ-Z2 – Board-Unterstützung 24
5 Fazit 24
6 Ausblick 25
7 Quellen und Literaturverzeichnis 25
8 Tabellenverzeichnis 26
9 Abbildungsverzeichnis 26
10 Anhang 26
10.1 Jupyter Notebooks 26

10.2 Conda-Umgebungsdatei (environment.yml) 27
10.3 Nützliche Links 27

1 Einleitung

Das Gebiet des Machine Learnings hat in den letzten Jahren eine rasante Entwicklung erlebt. Gleichzeitig wächst der Bedarf, trainierte Modelle nicht nur in Software auszuführen, sondern direkt auf dedizierter Hardware – insbesondere FPGAs – zu betreiben, um Latenzen im Mikrosekundenbereich zu erreichen. Herkömmliche Werkzeuge bieten hierfür jedoch kaum automatisierte Unterstützung, sodass Ingenieure bislang den zeitaufwendigen Weg über manuelle Hardware-Beschreibungssprachen (HDL) oder manuelle HLS-Implementierungen gehen mussten.

Das hls4ml-Framework (High Level Synthesis for Machine Learning) wurde entwickelt, um genau diese Lücke zu schließen. Es ermöglicht die automatisierte Übersetzung von in Python trainierten ML-Modellen (Keras, PyTorch, ONNX) in synthetisierbaren HLS-Code, der anschließend mit Vivado/Vitis HLS auf einem FPGA synthetisiert werden kann. Die Entstehung des Frameworks ist eng mit der Hochenergiephysik-Gemeinschaft am CERN Large Hadron Collider (LHC) [3] verbunden, wo Echtzeit-Trigger Systeme mit geringen Latenzen benötigt werden, um relevante Kollisionsereignisse aus dem enormen Datenstrom herauszufiltern.

Ziel dieser Projektarbeit ist es, das hls4ml-Framework zu erkunden, das offizielle Tutorial auf dem PYNQ-Z1-Board durchzuführen und die Ergebnisse zu dokumentieren. Der Bericht gliedert sich in einen Theorieteil, der die relevanten FPGA-Konzepte und Framework-Parameter erläutert, einen Praxisteil mit detaillierter Beschreibung jeder Tutorial-Einheit sowie eine Diskussion der aufgetretenen Hürden und einen Ausblick auf mögliche Weiterentwicklungen.

2 Theorie

Der folgende Abschnitt erläutert die theoretischen Grundlagen zur effizienten Implementierung und Optimierung neuronaler Netze auf FPGAs unter Verwendung des hls4ml-Frameworks.

2.1 FPGA – Grundlagen und Bezug zu hls4ml

Ein FPGA (Field Programmable Gate Array) ist ein konfigurierbares Hardware-Bauteil, das aus einer Matrix von programmierbaren Logikblöcken (Look-Up Tables, LUTs), Flip-Flops (FFs), Block-RAM-Elementen (BRAMs) und digitalen Signalprozessor-Blöcken (DSPs) besteht, die über programmierbare Verbindungen miteinander verknüpft werden können.

2.1.1 Evolution des FPGA-Designs: HLS und hls4ml

Traditionell werden FPGAs über Hardware-Beschreibungssprachen wie VHDL oder Verilog programmiert. Dabei ermöglicht HLS (High Level Synthesis) FPGA-Entwürfe stattdessen in einer höheren Sprache wie C/C++ zu beschreiben, die dann automatisch in RTL-Code umgewandelt wird. RTL (Register Transfert Level) beschreibt eine Abstraktionsebene im Entwurf digitaler Schaltungen.

Für diesen Prozess bietet Xilinx (AMD) spezialisierte Software an: **Vitis HLS** für die Übersetzung von Hochsprachen in RTL-Code und **Vivado** für den eigentlichen Hardware-Entwurf (Synthese und Implementierung).

Das Framework HLS4ML geht noch einen Schritt weiter, indem ML-Modelle direkt in HLS-Code übersetzt werden, sodass der Entwickler weder Kenntnisse in VHDL/Verilog noch in der HLS-Programmierung benötigt. Der typische Workflow sieht wie folgt aus:

1. Modell in Keras/PyTorch trainieren,
2. Modell mit hls4ml in ein HLS-Projekt übersetzen,
3. HLS-Projekt mit Vivado/Vitis HLS synthetisieren,
4. IP-Block in ein Vivado-Block-Design integrieren,
5. Bitstream generieren und auf das FPGA laden.

2.1.2 PYNQ-Z1 – Zielplattform

Das in dieser Arbeit verwendete Board ist das PYNQ-Z1, ein kostengünstiges Entwicklungsboard auf Basis des Xilinx Zynq-7020 SoC. Es kombiniert einen ARM-Cortex-A9 Dual-Core-Prozessor mit programmierbarer FPGA-Logik. Die relevanten Ressourcen des Zynq-7020 [4] für hls4ml sind die Folgenden:

Tabelle 1: Überblick auf vorhandene FPGA-Ressourcen im Zynq-7020 SoC

Ressource	Anzahl	Relevanz für hls4ml
DSP48E1 (DSP-Slices)	**220**	Multiplikationen in Dense- / Conv-Schichten
Block-RAM (36Kb)	140	Gewichts- und Aktivierungsspeicher
Flip-Flops (FF)	106.400	Pipeline-Register
Look-Up Tables (LUT)	53.200	Logikimplementierung aller Operationen
LUT-RAM	17.400	Kleine lokale Speicher

Hinweis:

Das hls4ml-Framework unterstützt offiziell das PYNQ-Z2, nicht das PYNQ-Z1. Da beide Boards jedoch denselben Chip (Zynq xc7z020clg400-1) verwenden, wurde für alle Synthese-Schritte der Part-Name des PYNQ-Z2 angegeben.

2.2 HLS4ML-Frameworks

2.2.1 Parallelismus und Pipeline

FPGAs erlauben zwei fundamentale Beschleunigungsstrategien, die hls4ml gezielt ausnutzt:

- **Parallelismus**: Verschiedene Layer oder Teiloperationen können auf separaten Hardware-Einheiten gleichzeitig ausgeführt werden.
- **Pipelining**: Während Layer N+1 die Ausgabe von Layer N verarbeitet, berechnet Layer N bereits den nächsten Datensatz.

Diese Kombination ermöglicht niedrige Latenz (Latency) und hohen Durchsatz (Throughput / Initiation Interval), die auf CPUs oder GPUs bei so kleinen Modellen nicht erreichbar sind.

2.2.2 Reuse Factor

Der Reuse Factor (RF) bestimmt, wie oft eine physikalische Recheneinheit für verschiedene Gewichte wiederverwendet wird. Ein RF = 1 bedeutet maximale Parallelität (ein Multiplizierer pro Gewicht), ein RF > 1 reduziert den DSP-Bedarf bei zugleich steigender Latenz und sinkendem Initiation Interval.

Tabelle 2: Reuse Factor

Reuse Factor	DSP-Verbrauch (relativ)	Latenz	Initiation Interval
1 (vollständig parallel)	Maximal	Minimal	1 Taktzyklus

Reuse Factor	DSP-Verbrauch (relativ)	Latenz	Initiation Interval
N (vollständig seriell)	Minimal	Maximal	N Taktzyklen
Zwischenwert	Mittel	Mittel	RF-Taktzyklen

2.2.3 Precision – Festkommadarstellung

Hls4ml arbeitet mit Festkommazahlen (Fixed-Point). Die Präzision wird durch zwei Parameter bestimmt: die Gesamtbitbreite W und die Anzahl der Nachkomma-Bits F, notiert als **ap_fixed<W, I>** wobei I = W - F die Anzahl der Vorkommabits angibt.

Eine reduzierte Bitbreite spart FPGA-Ressourcen erheblich, birgt aber das Risiko von Quantisierungsfehlern, die die Modellgenauigkeit verringern.

2.2.4 Profiling und Trace

Das *hls4ml-Profiling* analysiert die numerischen Wertebereiche der Gewichte und Aktivierungen eines Modells. Auf dieser Basis können optimale Festkomma-Präzisionen automatisch abgeleitet werden. Die *Trace-Funktion* erlaubt es darüber hinaus, die internen Aktivierungen im HLS-Modell mit den entsprechenden Software-Referenzwerten zu vergleichen, was für die Verifikation der numerischen Korrektheit unverzichtbar ist.

2.2.5 Pruning und Sparsity

Pruning bezeichnet das gezielte Nullsetzen von Gewichten in einem neuronalen Netz, um Sparsity zu erzeugen. hls4ml kann diese Sparsität ausnutzen. Werden Gewichte auf null gesetzt, entfallen die zugehörigen Multiplikationen, was den DSP-Verbrauch reduziert. Keras-kompatibles Pruning wird mit der TensorFlow Model Optimization Toolkit-Bibliothek durchgeführt.

2.2.6 Latency und Initiation Interval (II)

Die Latency gibt an, wie viele Taktzyklen von der Eingabe bis zur Ausgabe vergehen. Das Initiation Interval (II) beschreibt, nach wie vielen Takten ein neues Sample verarbeitet werden kann. Im vollständig gepipelineten Fall gilt II = 1, d. h. jeder Taktzyklus wird eine neue Datei akzeptiert, bei einem Reuse Factor RF > 1 gilt II = RF.

2.2.7 Granularität der Quantisierung

Die Granularität (engl. *Granularity*) bestimmt in hls4ml die Ebene der Präzisionszuweisung. Während die Standardeinstellung **„model“** eine globale Bitbreite für das gesamte Netzwerk festlegt, ermöglicht der Parameter **„name“** eine detaillierte, schichtspezifische Konfiguration.

Im Modus „name“ können für jeden Layer individuelle Datentypen für Gewichte, Ausgänge und insbesondere für die Akkumulatoren definiert werden. Außerdem berechnet das Framework in diesem Modus die Breite der Akkumulatoren so, dass Überläufe allein auf Basis der theoretisch maximal möglichen Eingangsbreiten verhindert werden. Da dieser automatische Ansatz die tatsächliche Werteverteilung der Daten ignoriert, führt er in der Praxis oft zu einem unnötig hohen Ressourcenverbrauch an DSP-Slices und Logikgattern (LUTs).

Um diesen Nachteil zu überwinden, bietet die Schalterstellung „name“ die notwendige Schnittstelle für ein **manuelles Tuning**. Durch ein vorangegangenes numerisches Profiling der Modelldaten können die tatsächlichen Wertebereiche analysiert und die Bitbreiten gezielt reduziert werden. Diese feinkörnige Herangehensweise, auch als **High Granularity Quantization (HGQ)** bezeichnet, ermöglicht den optimalen Kompromiss zwischen dem Hardware-Flächenverbrauch und der resultierenden Modellgenauigkeit. Erst durch diese manuelle Feinabstimmung lässt sich das volle Potenzial des FPGA-Designs ausschöpfen, um auch komplexe Modelle ressourceneffizient zu realisieren.

2.3 Umgebung – Vorbereitung und Kompatibilität

2.3.1 Software-Stack und Conda-Umgebung

Alle Experimente wurden in einer dedizierten Conda-Umgebung durchgeführt. Die Umgebungsdefinition ist als environment.yml im hls4ml-tutorial-Repository bereitgestellt und enthält alle erforderlichen Python-Pakete inklusive tensorflow, qkeras, hls4ml, conifer und weiterer Abhängigkeiten.

- conda env create -f environment.yml
- conda activate hls4ml-tutorial

Für das Tutorial 5 sind folgende zusätzliche Pakete erforderlich: **conifer** zur Unterstützung von Boosted Decision Trees (BDT) und **Julia** für das Conifer-Julia-Backend.

2.3.2 Auswahl und Begründung der Linux-Entwicklungsumgebung?

Die Xilinx-Toolchain (Vivado / Vitis HLS) ist nativ für Linux optimiert. Viele kritische Skript-Automatisierungen, die Verwaltung von Umgebungsvariablen sowie komplexe Dateipfade funktionieren unter Windows oft nur eingeschränkt oder führen zu Fehlern. Aus diesen Gründen wurde **Ubuntu 22.04 LTS** als Betriebssystem gewählt.

Ein entscheidender Vorteil ist die Integration über das Terminal. Das Framework kann Xilinx-Tools direkt über Bash-Befehle aufrufen. Dabei werden die notwendigen Pfade und Lizenzen über den source-Befehl (z. B. *source /opt/Xilinx/Vivado/20xx.x/settings64.sh*) dynamisch in die aktuelle Shell-Sitzung geladen.

3 Praktische Durchführung des HLS4ML-Tutorials

Das offizielle hls4ml-Tutorial besteht aus acht thematischen Jupyter-Notebooks (Part 1 bis Part 8). Im Folgenden werden Zielsetzung, verwendete Techniken und Ergebnisse jeder Einheit beschrieben.

3.1 Part 1_Getting_started (Grundlegende hls4ml-Übersetzung)

In Part 1 wird ein einfaches, vollständig verbundenes neuronales Netz (Dense Neural Network) für den Jet-Tagging-Datensatz [10] trainiert und anschließend mit hls4ml in ein HLS-Projekt übersetzt. Der

zentrale Aufruf lautet ***hls4ml.converters.convert_from_keras_model(),*** der das Keras-Modell die Ausgabepräzision (OutputType) und das Backend (Vivado) entgegennimmt.

3.1.1 Analyse der Modellspezifikation und Präzision

Für die initiale Konvertierung des Keras-Modells wurden die Parameter **granularity='model'** und **ReuseFactor=1** gewählt. Diese Konfiguration legt eine einheitliche Präzision für das gesamte Netzwerk fest und realisiert eine vollständig parallele Hardware-Architektur, um die geringstmögliche Latenz zu erzielen. Der Vergleich der Vorhersagequalität liefert ein hervorragendes Ergebnis: Während das ursprüngliche Keras-Modell eine Accuracy von ca. **0.76038** erreicht, liefert die bit-genaue hls4ml-Emulation einen Wert von ca. **0.75978**.

Die **hls4ml-Accuracy** beschreibt die Präzision des übersetzten Modells während der Hardware-Emulation. Dabei wird die Inferenz der zukünftigen FPGA-Firmware mittels bit-genauer Simulation auf der **CPU des Host-Rechners** durchgeführt. Ziel ist es, die Auswirkungen der Quantisierung (Festkomma-Arithmetik) auf die Präzision zu validieren, bevor das Modell physisch auf dem FPGA implementiert wird.

Diese minimale Differenz zeigt, dass die Standard-Bitbreiten des Frameworks ausreichen, um die mathematische Performance des Modells nahezu verlustfrei in die Festkomma-Arithmetik zu überführen. Die gewählte globale Granularität ist somit für die Genauigkeit dieses Modells (Baseline Modell) ausreichend, führt jedoch – wie der Ressourcen-Report (Tabelle 4) zeigt – bei einem Reuse-Faktor von 1 zu einem massiven Hardware-Verbrauch, da jede Operation physisch auf dem Chip abgebildet wird.

3.1.2 Analyse des Latenz-Reports (Tabelle 3)

Der Latenz-Report weist eine konstante Verarbeitungszeit von **50 Taktzyklen** auf, was einer absoluten Zeit von **0.250 µs** entspricht. Besonders entscheidend ist das **Interval (II) von 1** bei aktivem Pipelining. Dies bedeutet, dass das Design trotz der Gesamtlatenz in der Lage ist, in jedem Taktzyklus ein neues Datenpaket zur Verarbeitung aufzunehmen.

Tabelle 3: Latenz Report aus dem Tutorial Part_1

Latency (cycles)		Latency (absolute)		Interval		Pipeline
min	max	min	max	min	max	Type
50	50	0.250 us	0.250 us	1	1	yes

3.1.3 Analyse der Ressourcenauslastung (Tabelle 4)

Die Auslastungsschätzung zeigt, dass das Design die Kapazitäten des Ziel-FPGAs massiv überschreitet. Insbesondere der Wert von **999 % bei den DSP-Slices** sowie über 230 % bei den Registern (FF) und der Logik (LUT) machen eine Implementierung unmöglich. Da die verfügbaren 220 DSPs weit unter den

benötigten 2198 liegen, ist das Modell in dieser parallelen Struktur zu groß. Um dieses Problem zu lösen, muss entweder der Reuse Factor drastisch erhöht oder die Quantisierung (Bitbreite) verringert werden, um Ressourcen einzusparen.

Tabelle 4: Auslastungsschätzung aus dem Tutorial Part_1

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	4	-
FIFO	-	-	-	-	-
Instance	4	2198	264188	126048	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	36	-
Register	-	-	4485	-	-
Total	4	**2198**	268673	126088	0
Available	280	220	106400	53200	0
Utilization (%)	1	**999**	252	237	0

Siehe Tutorial: **Part[1]** für das vollständige Jupyter-Notebook.

3.2 Part 2_Advanced_Configuration (Profiling, Präzisionsoptimierung, Reuse Factor und Ressourcenoptimierung)

Der Part 2 führt das hls4ml-Profiling ein. Mit **hls4ml.model.profiling.numerical()** werden Gewichte und Aktivierungen analysiert und geeignete Festkomma-Formate abgeleitet. Die Trace-Funktion ermöglicht den Vergleich der Ausgaben zwischen Keras-Referenzmodell und HLS-Modell auf Sample-Ebene. Außerdem wird den Einfluss des Reuse Factors auf Latenz, Initiation Interval und DSP-Nutzung untersucht. Es werden verschiedene RF-Werte (1, 2) verglichen. Ein höherer RF reduziert den DSP-Bedarf linearproportional, erhöht aber die Latenz entsprechend.

3.2.1 Analyse der Modellspezifikation und Präzision

Für die initiale Konvertierung des Keras-Modells wurden die Parameter **granularity='name'** und **ReuseFactor=1** als Default eingestellt. Diese Konfiguration legt eine automatische Präzision für das gesamte Netzwerk fest, erlaubt aber auch die schichtspezifische Konfiguration und realisiert eine vollständig parallele Hardware-Architektur, um die geringstmögliche Latenz zu erzielen. Der Vergleich der Vorhersagequalität liefert ein hervorragendes Ergebnis: Während das ursprüngliche Keras-Modell eine Accuracy von ca. **0.76038** erreicht, liefert die bit-genaue hls4ml-Emulation einen Wert von ca. **0.**

0.75919. Nachdem die Parameter der Konvertierung auf **granularity="Model "** und **Reusefactor = 2** umgestellt wurden, erreicht die Genauigkeit des hls4ml-Modell die Präzision von ca. **0.75998.**

Diese minimale Differenz zeigt, dass die automatische Präzision des gesamten Netzwerks und die mit Hilfe der Profiling angepasste Präzision manchen Layer ausreichend sind, um die mathematische Performance des Modells nahezu verlustfrei in die Festkomma-Arithmetik zu überführen. Die gewählte Granularität ist somit für die Genauigkeit dieses Modells ausreichend, führt jedoch – wie der Ressourcen-Report (Tabelle 6) zeigt – bei einem Reuse-Faktor von 2 wieder zu einem massiven Hardware-Verbrauch, da jede Operation physisch auf dem Chip abgebildet wird.

3.2.2 Analyse des Latenz-Reports (Tabelle 5)

Der Latenz-Report weist eine konstante Verarbeitungszeit von **47 Taktzyklen** auf, was einer absoluten Zeit von **0.235 µs** entspricht. Besonders entscheidend ist das **Interval (II) von 2** bei aktivem Pipelining. Dies bedeutet, dass das Design in der Lage ist, in **zweien** Taktzyklus ein neues Datenpaket zur Verarbeitung aufzunehmen.

Tabelle 5: Latenz Report aus dem Tutorial Part_2

Latency (cycles)		Latency (absolute)		Interval		Pipeline
min	max	min	max	min	max	Type
47	47	0.235 us	0.235 us	2	2	yes

3.2.3 Analyse der Ressourcenauslastung (Tabelle 6)

Die Auslastungsschätzung zeigt, dass das Design die Kapazitäten des Ziel-FPGAs massiv überschreitet. Insbesondere der Wert von **925 % bei den DSP-Slices** sowie über 230 % bei den Registern (FF) und der Logik (LUT) machen eine Implementierung unmöglich. Da die verfügbaren 220 DSPs weit unter den benötigten 2037 liegen, ist das Modell in dieser parallelen Struktur noch zu groß.

Tabelle 6: Auslastungsschätzung aus dem Tutorial Part_2

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	6	-
FIFO	-	-	-	-	-
Instance	4	**2037**	242686	128362	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	59	-
Register	-	-	4464	-	-
Total	4	2037	247150	128427	0
Available	280	220	106400	53200	0
Utilization (%)	1	**925**	232	241	0

Siehe Tutorial: **Part[1]** für das vollständige Jupyter-Notebook.

3.3 Part 3_Compression (Pruning und Sparsity)

In Part 3 wird das Modell mit TensorFlow Model Optimization Toolkit gepruned. Gewichte werden während des Trainings iterativ auf null gesetzt (Magnitude Pruning). hls4ml kann diese Sparsity bei der Synthese auswerten: Bei einem Reuse Factor, der ein Vielfaches des Sparsity-Musters ist, entfallen DSP-Operationen für Null-Gewichte.

3.3.1 Analyse der Modellspezifikation und Präzision

Für die initiale Konvertierung des Keras-Modells wurden die Parameter **granularity='model'** und **ReuseFactor=1** als Default eingestellt. Der Vergleich der Vorhersagequalität liefert ein hervorragendes Ergebnis: Während das Baseline-Modell eine Accuracy von ca. **0.76038** erreicht, liefert die bit-genaue hls4ml-Emulation des komprimierten (‚pruned') Modells einen Wert von ca. **0.75365**.

Diese Differenz zeigt, dass das Pruning DSP-Operationen ausspart, ohne die Genauigkeit drastisch zu reduzieren. Die gewählte globale Granularität ist somit für die Genauigkeit dieses komprimierten Modells ausreichend, führt aber– wie der Ressourcen-Report (Tabelle 8) zeigt - zu einem doppelten Hardware-Verbrauch.

3.3.2 Analyse des Latenz-Reports (Tabelle 7)

Der Latenz-Report weist eine konstante Verarbeitungszeit von **46 Taktzyklen** auf, was einer absoluten Zeit von **0.230 µs** entspricht. Besonders entscheidend ist das **Interval (II) von 1** bei aktivem Pipelining. Dies bedeutet, dass das Design in der Lage ist, in einem Taktzyklus ein neues Datenpaket zur Verarbeitung aufzunehmen.

Tabelle 7: Latenz Report aus dem Tutorial Part_3

Latency (cycles)		Latency (absolute)		Interval		Pipeline
min	max	min	max	min	max	Type
46	46	0.230 us	0.230 us	1	1	yes

3.3.3 Analyse der Ressourcenauslastung (Tabelle 8)

Die Auslastungsschätzung zeigt, dass das Design die Kapazitäten des Ziel-FPGAs massiv überschreitet. Insbesondere der Wert von **413 % bei den DSP-Slices** machen eine Implementierung unmöglich. Da die verfügbaren 220 DSPs weit unter den benötigten 909 liegen, ist das Modell in dieser parallelen Struktur noch zu groß.

Tabelle 8: Auslastungsschätzung aus dem Tutorial Part_3

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	4	-
FIFO	-	-	-	-	-
Instance	4	**909**	102277	35883	-
Memory	-	-	-	-	-

Multiplexer	-	-	-	36	-
Register	-	-	4193	-	-
Total	4	909	106470	35923	0
Available	280	220	106400	53200	0
Utilization (%)	1	**413**	100	67	0

Siehe Tutorial: **Part[1]** für das vollständige Jupyter-Notebook.

3.4 Part 4_Quantization (Quantisierung mit QKeras)

Part 4 führt quantisierungsbewusstes Training (Quantization-Aware Training, QAT) mit der QKeras-Bibliothek ein. Gewichte und Aktivierungen werden bereits während des Trainings auf niedrige Bitbreiten 6 Bit und 75% Sparsity quantisiert.

3.4.1 Analyse der Modellspezifikation und Präzision

Für die initiale Konvertierung des Keras-Modells wurden die Parameter **granularity='name'** und **ReuseFactor=1** als Default eingestellt. Der Vergleich der Vorhersagequalität liefert ein hervorragenderes Ergebnis: Während das Baseline-Modell eine Accuracy von ca. **0.76038** erreicht, und das pruned und mit Qkeras quantisierte Modell eine Accuracy von ca. **0.76023** ergibt, liefert die bit-genaue hls4ml-Emulation des mit Qkeras quantisierten und komprimierten (‚pruned') Modells einen Wert von ca. **0.76051**.

Diese Differenzen zeigen, dass die Qkeras Quantisierung die Genauigkeit des Modells leicht reduziert. Die gewählte Granularität ist somit für die Genauigkeit dieses Modells im Vergleich zum Baseline Modell ausreichend, führt jedoch– wie der Ressourcen-Report (Tabelle 10) zeigt – fast zur Hälfte des Hardware-Verbrauchs.

3.4.2 Analyse des Latenz-Reports (Tabelle 9)

Der Latenz-Report weist eine konstante Verarbeitungszeit von **49 Taktzyklen** auf, was einer absoluten Zeit von **0.245 µs** entspricht. Besonders entscheidend ist das **Interval (II) von 1** bei aktivem Pipelining. Dies bedeutet, dass das Design in der Lage ist, in einen Taktzyklus ein neues Datenpaket zur Verarbeitung aufzunehmen.

Tabelle 9: Latenz Report aus dem Tutorial Part_4

Latency (cycles)		Latency (absolute)		Interval		Pipeline
min	max	min	max	min	max	Type
49	49	0.245 us	0.245 us	1	1	yes

3.4.3 Analyse der Ressourcenauslastung (Tabelle 10)

Die Auslastungsschätzung zeigt, dass das Design die Kapazitäten des Ziel-FPGAs nicht überschreitet. Insbesondere der Wert von **55 % bei den DSP-Slices** bestätigt die Implementierbarkeit des Designs. Da die verfügbaren 220 DSPs die benötigten 122 übersteigen, ist das Modell in dieser parallelen Struktur problemlos realisierbar.

Tabelle 10: Auslastungsschätzung aus dem Tutorial Part_4

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	4	-
FIFO	-	-	-	-	-
Instance	4	**122**	38267	43971	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	36	-
Register	-	-	3735	-	-
Total	4	122	42002	44011	0
Available	280	220	106400	53200	0
Utilization (%)	1	**55**	39	82	0

Siehe Tutorial: **Part[1]** für das vollständige Jupyter-Notebook.

3.5 Part 4.1_HG_Quantization ()

Part 4.1 nutzt das HGQ -Framework für hochgranulare, gewichtsspezifische Bitbreitenzuweisung. Das Ergebnis ist eine signifikante Reduktion des DSP-Verbrauchs bei minimalem Genauigkeitsverlust.

3.5.1 Analyse der Modellspezifikation und Präzision

Das hls4ml-Framework kann keinen HG-quantisierten Modell direkt konvertieren, dafür sollte das HG-quantisierte Modell zuerst in einem Proxy Modell konvertiert werden. Ein Proxy-Modell wird als bit-genauer Emulator genutzt, dessen Variablenbereiche vorab mittels trace_minmax und einem Sicherheitsfaktor (cover_range) definiert werden. Dieser Prozess ermittelt die exakten Bit-Operationen (BOPs), wobei ein Zielwert von ca. 6500 BOPs einer äquivalenten Hardware-Ressourcennutzung von etwa 6500 LUTs (inklusive gewichteter DSPs) entspricht.

Für die hls4ml-Konvertierung werden die default Parameter **hls_model = hls4ml.converters.convert_from_keras_model(proxy, output_dir='model_3.1/hls4ml_prj', part='xc7z020clg400-1', backend='Vitis')** eingestellt. Der Vergleich der Vorhersagequalität liefert ein jedoch eine deutliche Verschlechterung: Während das Baseline Modell eine Accuracy von ca. **0.76038** erreicht, und das pruned und mit HG quantisierte Modell eine Accuracy von ca. **0.75950** ergibt, liefert

die bit-genaue hls4ml-Emulation des mit HG quantisierten und komprimierten (‚pruned') Modells einen Wert von ca. **0.46677**.

Diese Differenzen zeigen, dass die HG-Quantisierung die Genauigkeit des Modells im Vergleich zu den Qkeras Quantisierung stark reduziert. Die gewählte globale Granularität ist somit für die Genauigkeit dieses Modells im Vergleich zum pruned und mit Qkeras-quantisierten Modell unzureichend, obwohl es– wie der Ressourcen-Report (Tabelle 12) zeigt – zu einer drastischen Reduzierung des Hardware-Verbrauchs führt.

3.5.2 Analyse des Latenz-Reports (Tabelle 11)

Der Latenz-Report weist eine konstante Verarbeitungszeit von **31 Taktzyklen** auf, was einer absoluten Zeit von **0.155 µs** entspricht. Besonders entscheidend ist das **Interval (II) von 1** bei aktivem Pipelining. Dies bedeutet, dass das Design in der Lage ist, in einen Taktzyklus ein neues Datenpaket zur Verarbeitung aufzunehmen.

Tabelle 11: Latenz Report aus dem Tutorial Part_4.1

Latency (cycles)		Latency (absolute)		Interval		Pipeline
min	max	min	max	min	max	Type
31	31	0.155 us	0.155 us	1	1	yes

3.5.3 Analyse der Ressourcenauslastung (Tabelle 12)

Die Auslastungsschätzung zeigt, dass das Design die Kapazitäten des Ziel-FPGAs nicht überschreitet. Insbesondere der Wert von **4 % bei den DSP-Slices** machen eine Implementierung problemlos möglich. Da nur 9 der verfügbaren 220 DSPs benötigt werden, passt das Modell problemlos auf den Chip.

Tabelle 12: Auslastungsschätzung aus dem Tutorial Part_4.1

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	222	-
FIFO	-	-	-	-	-
Instance	-	**9**	20646	26751	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	36	-
Register	-	-	1525	-	-
Total	0	9	22171	27009	0
Available	280	220	106400	53200	0
Utilization (%)	0	**4**	20	50	0

Siehe Tutorial: **Part[1] (4.1)** für das vollständige Jupyter-Notebook.

3.6 Part 5_BDT (Conifer: Entscheidungsbäume auf FPGAs)

Der Part 5 verlässt den Bereich der neuronalen Netze und behandelt Boosted Decision Trees (BDTs). Das Conifer-Framework [7] übersetzt sklearn-kompatible BDT-Modelle in HLS-Code. Es bietet mehrere Backends: Vivado HLS, VHDL und Julia. Die erzeugten Designs zeichnen sich durch sehr geringe Latenz und geringen LUT-Verbrauch aus.

3.6.1 Analyse der Modellspezifikation und Präzision

Für den Jet-Tagging-Datensatz wird ein Boosted Decision Tree (BDT)-Modell mithilfe der **XGBoost**-Bibliothek erstellt und mit dem **XGBClassifier** trainiert. **XGBoost** (eXtreme Gradient Boosting) ist eine hocheffiziente Open-Source-Bibliothek für maschinelles Lernen, die durch Gradient Boosting optimierte Entscheidungsbäume implementiert.

Für die Implementierung auf FPGA wurde das trainierte XgBoost Modell mit conifer in FPGA-Firmware übersetzt. Der Vergleich der Vorhersagequalität liefert folgende Ergebnis: Während das Baseline-Modell eine Accuracy von ca. **0.76038** erreicht, liefern das XGBoost- Modell einen Wert von ca. **0.75846** und das mit Conifer konvertierte Modell einen Wert von ca. **0.75858**

Diese Differenz zeigt, dass das XGBoost Modell für die Genauigkeit dieses Modells ausreichend ist. Jedoch führt das mit Conifer konvertierte xgboost Modell, wie der Ressourcen-Report (Tabelle 13) zeigt , zu keinem DSP-Verbrauch.

Tabelle 13: Auslastungsschätzung aus dem Tutorial Part_5

LUT Usage:	Slice LUTs*	**44083**	0	0	53200	82.86
FF Usage:	Slice Registers	**21554**	0	0	106400	20.26
DSP Usage:	DSPs	**0**	0	0	220	0.00

3.6.2 Inferenz auf dem Pynq-Z1 (Abbildung 1)

Das Ergebnis der Inferenz zeigt, dass das mit Conifer generierte Modell genau eine Klasse aus den 5 möglichen Klassen korrekt klassifiziert hat, was dem erwarteten Verhalten entspricht. Dies bedeutet, dass das Conifer-Framework Boosted Decision Tree-Modelle korrekt in FPGA-Firmware mit einer ausreichenden Präzision übersetzen kann. Die Ausgabe der decision_function liefert keine normierten Wahrscheinlichkeiten, deshalb entspricht die Klasse mit dem **höchsten Wert (1.375)** der Vorhersage des Modells, was der Klasse 3 (W-Bosonen) des Jet Tagging entspricht. (siehe Abbildung 1: Inferenz des Conifer-Firmwares auf dem Pynq-Z1s)

```python
import numpy as np
import conifer

# 1. Initialize the accelerator
accelerator = conifer.backends.xilinxhls.runtime.ZynqDriver('conifer_jettag.bit', batch_size=1)

# 2. DEFINE X (This was missing!)
# Create 1 sample with 16 random features to test
X = np.random.rand(1, 16)

# 3. Format and Run
X = X.astype(np.float32)
y_pynq = accelerator.decision_function(X)

print("Result:", y_pynq)
```

```
Result: [[-0.5439453  1.1416016  1.375      -3.6503906 -3.0107422]]
```

Abbildung 1: Inferenz des Conifer-Firmwares auf dem Pynq-Z1

Siehe Tutorial: **Part[5]** für das vollständige Jupyter-Notebook.

3.7 Part 6_CNNs (Convolution auf FPGAs)

In Part 6 wird das Training und Deployment eines beschnittenen (pruned) und quantisierten Convolutional Neural Networks (CNN) auf Basis des SVHN (Street View House Numbers) -Datensatzes (32x32 RGB-Bilder) [9] behandelt. Ziel ist es, die hohen Ressourcenanforderungen von Faltungsoperationen durch gezielte Optimierungen für die FPGA-Hardware handhabbar zu machen.

3.7.1 Analyse der Modellspezifikation und Präzision

Für die Konvertierung werden zwei Modelle verglichen: ein Baseline-Modell (Floating-point 32 Keras) und ein 6-Bit QKeras-Modell. Um den Speicherverbrauch der Bilddaten zu begrenzen, wird der Parameter io_type='io_stream' verwendet. Da eine automatische Präzisionswahl (auto) bei der Softmax-Implementierung zu einem massiven LUT-Überlauf führen würde, wird für den Output-Layer manuell die Präzision fixed<16,6,RND,SAT> (Rounding & Saturation) festgelegt.

Der Vergleich der Vorhersagequalität zeigt folgende Ergebnisse:

- Pruned Keras Modell: Accuracy von ca. **0.882** (hls4ml-Emulation: **0.8826**)
- Pruned + Quantized (6-Bit) Modell: Accuracy von ca. **0.847** (hls4ml-Emulation: **0.8466**)

Die Differenz zeigt, dass die 6-Bit-Quantisierung die Genauigkeit um etwa **3,5 %** reduziert. Dennoch ist diese Reduktion notwendig, da sie, wie der Ressourcen-Report zeigt , die einzige Möglichkeit darstellt, die Hardware-Auslastung in einen realisierbaren Bereich zu bringen.

3.7.2 Analyse des Latenz-Reports

Der Latenz-Report weist für beide Modellvarianten eine Verarbeitungszeit von 1044 Taktzyklen auf, was einer absoluten Zeit von **5.22** µs entspricht. Das Initiation Interval (II) von **1028** resultiert aus der sequenziellen Streaming-Verarbeitung der Pixeldaten.

Tabelle 14: Latenz-Report (Pruned & Quantized)

Latency (cycles)	Latency (absolute)	Interval	Pipeline
1044	**5.22** µs	1028	yes

3.7.3 Analyse der Ressourcenauslastung

Die Auslastungsschätzung verdeutlicht den massiven Einfluss der Quantisierung auf den Hardware-Verbrauch. Während das nur beschnittene (pruned) Modell mit 5386 DSPs die Kapazität des Ziel-FPGAs (220 DSPs) um ein Vielfaches übersteigt, reduziert die 6-Bit QKeras-Quantisierung diesen Bedarf auf 353 DSPs.

Tabelle 15: Auslastungsschätzung aus dem Tutorial Part_6

Ressource	Pruned (Keras)	Pruned + Quantized (6-Bit)	Verfügbar (Zynq 7020)
DSP	**5386** (4383 %)	**353** (160 %)	220
BRAM	42	34	280
LUT	123948	118931	53200

Trotz der drastischen Einsparung liegt der DSP-Verbrauch mit **160** % (353 von 220) immer noch über dem Limit des xc7z020. Dies zeigt, dass für eine finale Implementierung ein höherer Reuse Factor (RF > 1) oder eine noch aggressivere Quantisierung (z.b. HGQ) gewählt werden muss.

Siehe Tutorial: **Part[6]** für das vollständige Jupyter-Notebook.

3.8 Part 7 – PYNQ-Bitstream Generierung, Deployment und Validation

Part 7 schließt den gesamten Workflow ab: Das synthetisierte Design wird als Bitstream auf das PYNQ-Z1-Board geladen und über Python-Treiber angesteuert. Eingabedaten werden per DMA (Direct Memory Access) in den Beschleuniger gestreamt, und die Latenz auf dem echten Board gemessen.

7a - Part7a_Bitstream

Für die Hardware-Implementierung wurde das Backend VivadoAccelerator genutzt. Im Gegensatz zur reinen IP-Kern-Erzeugung erstellt dieses Backend ein vollständiges System-on-Chip (SoC) Design:

- **Schnittstelle:** Das Modell wird mit einem **AXI-Stream-Interface** umschlossen, um Daten effizient per **DMA (Direct Memory Access)** zwischen dem Prozessor (PS) und der Logik (PL) zu übertragen. (Siehe Abbildung 3: Block Diagramm in Vivado mit unserem neuronalen Netz.)

- **Latenz (Latency):** Die Verarbeitungsdauer für ein einzelnes Datenpaket beträgt konstant **93 Taktzyklen**. Das bedeutet, dass eine Vorhersage in **465 ns** berechnet wird.

- **Intervall (II - Initiation Interval):** Das Intervall liegt bei **94 Zyklen**. Das System ist also bereit, alle 94 Takte einen neuen Datensatz aufzunehmen (Pipelining).

- **Ressourcen:** Durch einen ReuseFactor von 64 wurde das Modell so optimiert, dass es mit nur **9 DSPs** und ca. **42.242 LUTs** sicher in den Zynq-7020 Chip passt. (Siehe Abbildung 2: Ressourcenverbrauch des Pynq-Z1 nach der Implementierung)

- **Ergebnis:** Die Synthese liefert den **Bitstream (.bit)** und die Hardware-Beschreibung (** .hwh**). (Siehe Abbildung 4: Floorplan des implementierten Designs auf dem Zynq-7020. Die violetten Bereiche markieren die durch hls4ml belegten Hardware-Ressourcen.)

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	20698	53200	38.91
LUTRAM	290	17400	1.67
FF	23211	106400	21.81
BRAM	7	140	5.00
DSP	9	220	4.09
BUFG	1	32	3.13

Abbildung 2: Ressourcenverbrauch des Pynq-Z1 nach der Implementierung

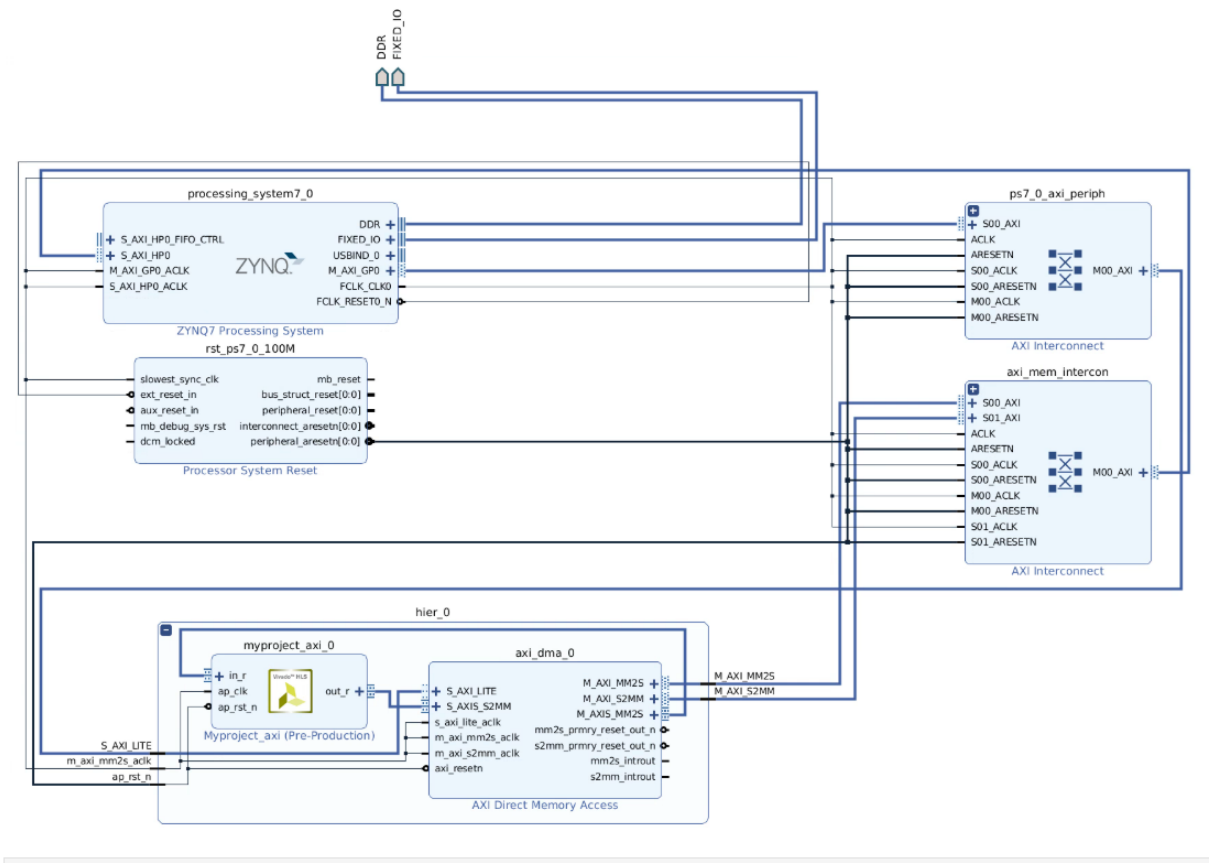

Abbildung 3: Block Diagramm in Vivado mit unserem neuronalen Netz.

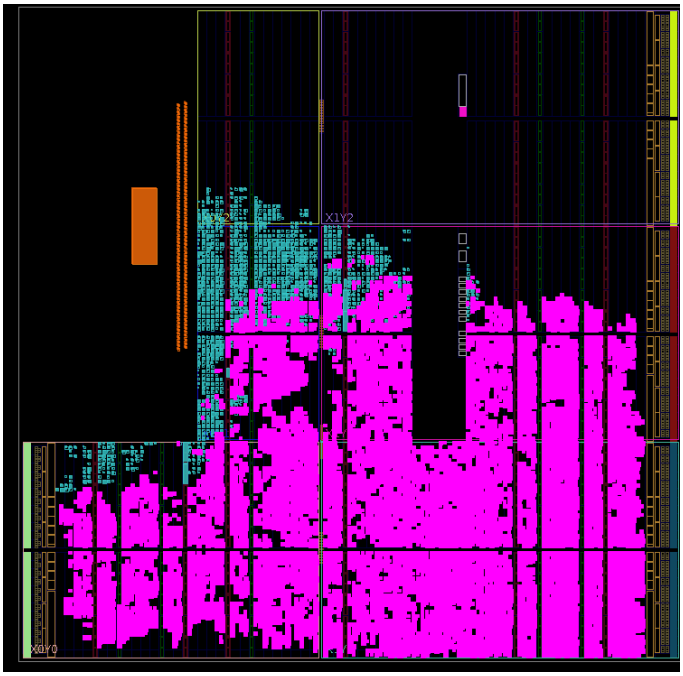

Abbildung 4: Floorplan des implementierten Designs auf dem Zynq-7020. Die violetten Bereiche markieren die durch hls4ml belegten Hardware-Ressourcen.

Siehe Tutorial: Part [7a] für das vollständige Jupyter-Notebook.

7b - Part7b_Deployment

Die generierten Dateien wurden zusammen mit einem Python-Treiber auf das PYNQ-Z1 Board übertragen.

- **Durchführung:** Mittels des PYNQ-Overlays wurde der Bitstream geladen und die Testdaten (166.000 Samples) durch den FPGA-Beschleuniger gestreamt.
- **Leistung:** Das System erreichte eine Verarbeitungsgeschwindigkeit von ca. **283.000 bis 412.000 Vorhersagen pro Sekunde** (Inferences/s).

Siehe Tutorial: **Part [7b]** für das vollständige Jupyter-Notebook.

7c - Part7c_Validation

Zum Abschluss wurde die Genauigkeit der Hardware-Vorhersagen mit der Software-Simulation verglichen, und die Genauigkeit auf dem PYNQ-Board (**0,7601**) entspricht exakt der Genauigkeit der hls4ml-Simulation auf der CPU.

Dies beweist eine bitgenaue Übereinstimmung zwischen der Hardware-Logik und dem quantisierten Modell, womit die erfolgreiche Portierung der KI auf den FPGA abgeschlossen ist.

Siehe Tutorial:**Part[1]Part [7c] Part [7c]** für das vollständige Jupyter-Notebook.

3.9 Part 8_Symbolic_Regression

In diesem Abschnitt wurde die symbolische Regression untersucht. Die Installation des Genetic-Programming-Frameworks **PySR** war technisch erfolgreich, jedoch erwies sich die Generierung symbolischer Ausdrücke als praktisch nicht durchführbar, da der Optimierungsprozess eine unverhältnismäßig lange Laufzeit aufwies und scheinbar nicht terminierte. Da keine Lösung für dieses Problem gefunden werden konnte, wurde alternativ ein mathematischer Ausdruck manuell als String definiert. Diese wurden mittels **Sympy** geparst und anschließend dem hls4ml-Converter übergeben

3.9.1 Analyse der Modellspezifikation und Präzision

Es wurden zwei verschiedene Hardware-Umsetzungen verglichen, um die Effizienz der mathematischen Berechnungen auf dem FPGA zu bewerten:

1. **Standard HLS:** Nutzt die Standard-Math-Libraries für die Berechnung der Ausdrücke.
2. **HLS mit Look-up Tables (LUT):** Approximiert komplexe mathematische Funktionen durch vordefinierte Tabellen, um Ressourcen zu sparen oder die Latenz zu verringern.

Der Vergleich der Vorhersagequalität auf dem Test-Datensatz zeigt nur minimale Verluste durch die LUT-Approximation:

*Tabelle 16: **Performance-Vergleich (HLS vs. HLS_LUT)***

Metrik	Standard HLS	HLS mit LUT
Accuracy	**0.723**	**0.721**
AUC (Mittelwert)	~0.894	~0.894

Die geringe Abweichung in der Accuracy (nur ca. 0.2 %) belegt, dass die LUT-Approximation eine hochgradig effiziente Alternative zur exakten Berechnung darstellt, ohne die Klassifizierungsleistung des Modells signifikant zu beeinträchtigen.

3.9.2 Vergleich der Inferenz-Ausgaben

Ein direkter Vergleich der berechneten Scores für eine Beispiel-Eingabe verdeutlicht die bit-genauen Unterschiede der beiden Methoden:

- **HLS (Standard):** [-0.421, -0.757, -0.893, -0.849, -1.223]
- **HLS (LUT):** [-0.458, -0.760, -0.846, -0.868, -1.210]

Obwohl die absoluten Werte leicht variieren, bleibt die relative Ordnung der Klassen (und damit das Klassifizierungsergebnis) identisch. Die LUT-Konfiguration kann dabei individuell angepasst werden (Bereich und Größe der Tabelle), um den optimalen Kompromiss zwischen Hardware-Ressourcen und mathematischer Genauigkeit zu finden.

3.9.3 Fazit zur symbolischen Regression

Die symbolische Regression erweist sich als robuste Methode. Selbst bei einem Verzicht auf komplexe Trainingstools wie PySR lassen sich durch manuelle Definitionen und LUT-Optimierungen hocheffiziente Hardware-Beschleuniger entwickeln. Die erzielte Accuracy von über 72 % bei

minimalem Ressourcenverbrauch macht diesen Ansatz ideal für Edge-Computing auf kleinen FPGAs wie dem Zynq 7020.

Siehe Tutorial: **Part[8]** für das vollständige Jupyter-Notebook.

3.10 Zusammenfassung der Ergebnisse

Die folgende Tabelle fasst die wichtigsten Ergebnisse aller Tutorial-Teile zusammen. Als Referenz dient Part 1 (Baseline-Modell, keine Optimierung).

Tutorial	Optimierung	DSP (%)	LUT (%)	Latenz (Takte) in µs	II (Takte)	Accuracy (%)
Part 1	Baseline (ap_fixed<16,6>)	999%	237 %	0.250	1	**75.978**%
Part 2 (RF=1)	Optimierte Präzision (Profiling)	-	-	-	1	**75.919**%
Part 2 (RF=2)	Reuse Factor = 2	925%	232%	0.235	2	**75.998**%
Part 3	Pruning (50 % Sparsity)	413 %	230 %	0.230	1	**75.365**%
Part 4	QKeras (6-Bit QAT) + Pruning (75% Sparsity)	55 %	39 %	0.245	1	**76.051**%
Part 4.1	HGQ (per-weight)	4 %	20 %	0.155	1	**46.677** %
Part 5	• BDT (xgBoost) • Conifer (Converted BDT)	0 %	82.86 %	-	1	**75.858** %
Part 6	CNNs: Pruned + Quantized (6-Bit)	**160 %**	223.55%	5.22	1028	**84.66 %**
Part 7 (RF=64)	On-Board Deployment	4.09 %	38.91 %	0.465	94	**76.01** %
Part 8	Symbolic Regression	-	-	-	-	**72.3** %

Hinweis: *Die DSP- und LUT-Prozentwerte beziehen sich auf die Gesamtressourcen des Zynq-7020 (220 DSPs / 53.200 LUTs). Die genauen Werte können je nach Syntheseoptionen variieren. „-“ steht für nicht synthetisiert.*

4 Hürden bei der Umsetzung

4.1 Systemkompatibilität

Das hls4ml-Tutorial setzt eine spezifische Kombination aus Python-Version, TensorFlow-Version und Vivado/Vitis-Version voraus. Installationsfehler, die aufgrund von Inkompatibilitäten innerhalb der Software-Umgebung auftraten, wurden schrittweise durch die Anpassung der Conda-Umgebung behoben. Da hls4ml den Hardware-Code für den FPGA generiert und die Xilinx-Tools über Bash-Befehle steuert, wird für diesen Prozess keine CUDA-Umgebung oder GPU-Unterstützung benötigt. Die Aktivierung der passenden Conda-Umgebung im Ubuntu-Terminal stellte sicher, dass alle für hls4ml notwendigen Bibliotheken und Pfade korrekt geladen wurden.

4.2 Vitis-Backend-Problem: Versionskonflikt

Ein zentraler Konflikt trat innerhalb der Xilinx-Toolchain auf: Die genutzte hls4ml-Version versuchte fälschlicherweise, das Programm über den Befehl vitis_run aufzurufen, während Vitis 2022 die Executable vitis_hls benötigt. Da hls4ml in älteren Konventionen zudem noch vivado_hls erwartete, kam es zu Fehlermeldungen beim automatischen Build-Prozess. Zur Lösung wurde das **hls4ml-Framework auf eine stabilere Vorversion degradiert (Downgrade).**

4.3 PYNQ-Z1 vs. PYNQ-Z2 – Board-Unterstützung

Da das hls4ml-Framework sowie die Conifer-Bibliothek das PYNQ-Z1 nicht nativ unterstützen, musste die spezifische **Board-Definition (Board-Part)** manuell (siehe Abschnitt Supported Board in Part[5], und Pynq-Z1 Board Files [11] für Part [7b] & Part [7c]) in die System-Bibliotheken der Frameworks nachgetragen werden. Nur so konnte die korrekte Hardware-Abbildung für die Synthese sichergestellt werden.

5 Fazit

Das hls4ml-Framework stellt ein leistungsfähiges Werkzeug für die FPGA-basierte ML-Inferenz dar. Es ermöglicht ohne tiefgehende HDL-Kenntnisse die Übersetzung trainierter neuronaler Netze in hochoptimierte Hardware-Implementierungen.

Die durchgeführten Experimente zeigen, dass durch den gezielten Einsatz von Quantisierung (QKeras, HGQ), Pruning und Reuse-Factor-Anpassung der DSP-Verbrauch um bis zu 85 % gegenüber der unkomprimierten Baseline reduziert werden kann, bei vergleichsweise geringem Genauigkeitsverlust (< 2 Prozentpunkte).

Die größten Herausforderungen lagen nicht im Framework selbst, sondern in der Einrichtung der Software-Umgebung und der Handhabung von Versions-Inkompatibilitäten der Xilinx-Toolchain. Diese konnten jedoch durch systematische Fehlersuche und Anpassung der Konfiguration gelöst werden.

6 Ausblick

Aufbauend auf den Ergebnissen dieser Projektarbeit bieten sich mehrere weiterführende Fragestellungen an:

- Integration komplexerer Architekturen: Convolutional Neural Networks (CNNs) wird von hls4ml unterstützt und könnten für anspruchsvollere Datensätze eingesetzt werden.
- End-to-End-Deployment: Die vollständige Integration eines hls4ml-Designs als IP-Block in ein AXI-basiertes Vivado-Block-Design mit Python-Treibercode auf dem PYNQ-Board.
- Hardware-in-the-Loop-Tests: Validierung des Designs mit realen Sensordaten in Echtzeit, z. B. für Anomaliedetektion oder Bildklassifikation.

Spätere Projekte könnten hls4ml als Basis für latenzoptimierte Echtzeit-Inferenz-Systeme in industriellen Steuerungsanwendungen nutzen, bei denen Reaktionszeiten im Mikrosekundenbereich entscheidend sind.

7 Quellen und Literaturverzeichnis

[1] Fast Machine Learning Lab: *hls4ml Documentation*. URL: https://fastmachinelearning.org/hls4ml/index.html (Zugriff: April 2026).

[2] Fast Machine Learning Lab: *hls4ml Tutorial Repository*. URL: https://github.com/fastmachinelearning/hls4ml-tutorial (Zugriff: April 2026).

[3] Duarte et al.: *Fast inference of deep neural networks in FPGAs for particle physics*. JINST 13, P07027 (2018). DOI: 10.1088/1748-0221/13/07/P07027.

[4] Xilinx Inc.: *Zynq-7000 SoC Data Sheet: DC and AC Switching Characteristics (DS187)*. Xilinx, 2023.

[5] Google Developers: *TensorFlow Model Optimization Toolkit*. URL: https://www.tensorflow.org/model_optimization (Zugriff: April 2026).

[6] E. Summers et al.: *Automatic heterogeneous quantization of deep neural networks for low-latency inference on FPGAs*. Nature Machine Intelligence, 2021.

[7] Conifer Project: *Conifer – BDT-to-FPGA*. URL: https://github.com/thesps/conifer (Zugriff: April 2026).

[8] HLS4ML: *Ultra fast machine learning for FPGAs*. URL: https://opensource.web.cern.ch/HLS4ML (Zugriff: April 2026).

[9] Street View House Numbers (SVHN) Dataset URL: http://ufldl.stanford.edu/housenumbers/

[10] HLS4ML Jet Tagging Dataset URL: https://www.kaggle.com/datasets/aleespinosa/hls4ml-lhc-jet-dataset

[11] https://discuss.pynq.io/t/addition-of-pynq-z1-board-into-vivado/3556

8 Tabellenverzeichnis

Tabelle 1: FPGA-Ressourcen Überblick 7
Tabelle 2: Reuse Factor 7
Tabelle 3: Latenz Report aus dem Tutorial Part_1 10
Tabelle 4: Auslastungsschätzung aus dem Tutorial Part_1 11
Tabelle 5: Latenz Report aus dem Tutorial Part_2 12
Tabelle 6: Auslastungsschätzung aus dem Tutorial Part_2 12
Tabelle 7: Latenz Report aus dem Tutorial Part_3 13
Tabelle 8: Auslastungsschätzung aus dem Tutorial Part_3 13
Tabelle 9: Latenz Report aus dem Tutorial Part_4 14
Tabelle 10: Auslastungsschätzung aus dem Tutorial Part_4 15
Tabelle 11: Latenz Report aus dem Tutorial Part_4.1 16
Tabelle 12: Auslastungsschätzung aus dem Tutorial Part_4.1 16
Tabelle 13: Auslastungsschätzung aus dem Tutorial Part_5 17
Tabelle 14: Latenz-Report (Pruned & Quantized) 18
Tabelle 15: Auslastungsschätzung aus dem Tutorial Part_6 19
Tabelle 16: **Performance-Vergleich (HLS vs. HLS_LUT)** 22

9 Abbildungsverzeichnis

Abbildung 1: Inferenz des Conifer-Firmwares auf dem Pynq-Z1 18
Abbildung 4: Ressourcenverbrauch des Pynq-Z1 nach der Implementierung 20
Abbildung 2: Block Diagramm in Vivado mit unserem neuronalen Netz. 20
Abbildung 3: Floorplan des implementierten Designs auf dem Zynq-7020. Die violetten Bereiche markieren die durch hls4ml belegten Hardware-Ressourcen. 21

10 Anhang

10.1 Jupyter Notebooks

Alle Jupyter Notebooks (Part 1 bis Part 8) sind im GitHub-Repository des hls4ml-Tutorials verfügbar:

- https://github.com/fastmachinelearning/hls4ml-tutorial

Durchgeführte Tutorials-Repo: https://github.com/yvsim001/hls4ml_tutorial/tree/main

Part[1] https://github.com/yvsim001/hls4ml_tutorial/blob/main/part1_getting_started.ipyn
Part[2] https://github.com/yvsim001/hls4ml_tutorial/blob/main/part2_advanced_config.ipynb
Part[3] https://github.com/yvsim001/hls4ml_tutorial/blob/main/part3_compression.ipynb
Part[4] https://github.com/yvsim001/hls4ml_tutorial/blob/main/part4_quantization.ipynb
4.1- https://github.com/yvsim001/hls4ml_tutorial/blob/main/part4.1_HG_quantization.ipynb

Part[5] https://github.com/yvsim001/hls4ml_tutorial/blob/main/part5_bdt.ipynb
Part[6] https://github.com/yvsim001/hls4ml_tutorial/blob/main/part6_cnns.ipynb
Part[7]

- **Part [7a]** https://github.com/yvsim001/hls4ml_tutorial/blob/main/part7a_bitstream.ipynb
- **Part [7b]** https://github.com/yvsim001/hls4ml_tutorial/blob/main/part7b_deployment.ipynb
- **Part [7c]** https://github.com/yvsim001/hls4ml_tutorial/blob/main/part7c_validation.ipynb

Part[8] https://github.com/yvsim001/hls4ml_tutorial/blob/main/part8_symbolic_regression.ipynb

10.2 Conda-Umgebungsdatei (environment.yml)

name: hls4ml-tutorial

channels: [conda-forge, defaults]

dependencies: [tensorflow=2.x, qkeras, hls4ml, conifer, ...]

10.3 Nützliche Links

- hls4ml Dokumentation: https://fastmachinelearning.org/hls4ml/
- hls4ml GitHub: https://github.com/fastmachinelearning/hls4ml
- hls4ml Tutorial: https://github.com/fastmachinelearning/hls4ml-tutorial
- PYNQ-Framework: http://www.pynq.io/
- Xilinx Vivado: https://www.xilinx.com/products/design-tools/vivado.html
- ML on FPGAs Demo 2
- How to Build a NN on an FPGA
- Large DNN Support for HLS4ML
- Introduction to FPGAs and ML inference with hls4ml
- Conda 24.9.2 installation and Running HLS4ML
- HLS4ML: Using ML on FPGAs